ChatGPT disse:

Your model configuration is a good starting point for a comparative credit-risk study. You have selected six algorithms that approach the classification problem in different ways.

The most important thing to understand is that **each algorithm learns patterns differently, so the same hyperparameter can have different meanings across models, and not every model has the same controls.**

I'll go through the models, explain every hyperparameter in your configuration, and then discuss what I would add, what I would leave out, and why.

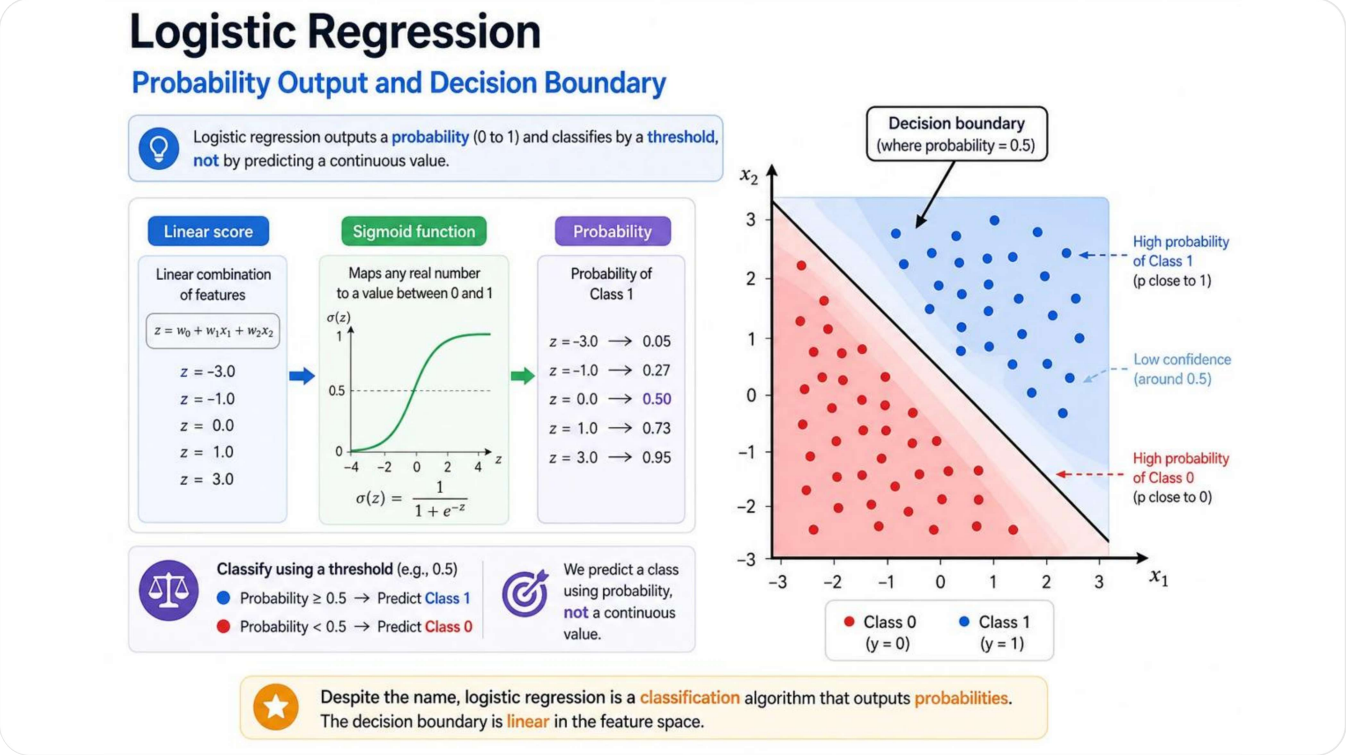
1. First, understand the six algorithms

Your models fall into four broad families:

Family	Models	How they learn
Linear	Logistic Regression	Finds a weighted relationship between input variables and default probability.
Single tree	Decision Tree	Creates decision rules based on customer characteristics.
Bagging / ensemble trees	Random Forest, Extra Trees	Combines multiple trees to improve stability and predictive performance.
Boosting	Gradient Boosting, XGBoost	Builds trees sequentially, with each new tree correcting errors made by previous trees.

Let's understand them individually.

1. Logistic Regression



Despite its name, Logistic Regression is a classification algorithm.

It estimates the probability that an observation belongs to a particular class.

For your project:

$$P(Default = 1) = \frac{1}{1 + e^{-z}}$$

Where z is a weighted combination of the input variables.

For example, the model may learn that:

- Higher loan-to-income ratios increase default probability.
- Longer credit history may decrease default probability.
- Higher interest rates may increase default probability.

The model estimates these relationships from your training data.

Main advantage: Easy to interpret, computationally efficient, and useful as a baseline.

Main limitation: It primarily captures linear relationships in the transformed feature space. Complex interactions and nonlinear relationships may require feature engineering.

Your hyperparameters

Hyperparameter	Your values	What it controls
`C`	0.01, 0.1, 1, 10, 100	Controls regularization strength.
`solver`	lbfgs	Optimization algorithm used to estimate model coefficients.

`C` — Regularization strength

This is the inverse of regularization strength.

- Small `C` : Stronger regularization, smaller coefficients, simpler model.
- Large `C` : Weaker regularization, more freedom to fit the training data.

Regularization helps prevent overfitting.

For example:

- `C=0.01` : Strong restriction on coefficient magnitude.
- `C=100` : Much weaker restriction.

Your search range is sensible because it explores both strongly and weakly regularized models.

`solver="lbfgs"`

Specifies the numerical optimization algorithm used to find the model coefficients.

Why use it?

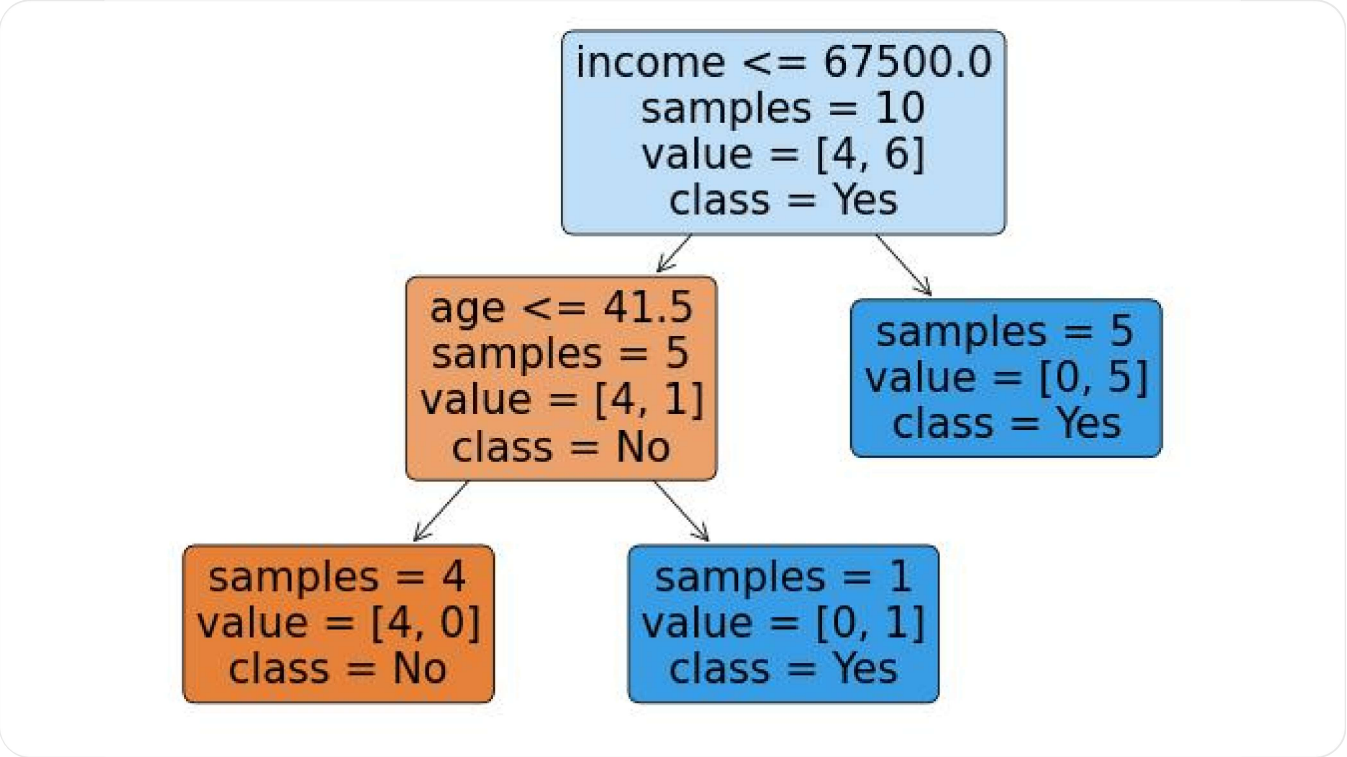
- Works well for L2 regularization.
- Supports binary classification.
- Is a good general-purpose solver for datasets of moderate size.

What you didn't include:

- `penalty` : You are using the default, L2 regularization. I would keep it initially.
- `class_weight` : Already set to `"balanced"` in the model definition.
- `tol` : Optimization convergence tolerance. The default is usually adequate.
- `fit_intercept` : Whether to estimate an intercept. The default is `True` , which I would keep.

One important consideration: Logistic Regression generally benefits from `StandardScaler` , unlike tree-based models.

2. Decision Tree



A Decision Tree learns a sequence of decision rules.

Imagine your model learning something like:

```
Is loan-to-income ratio > 0.35?
│
├── Yes
│   └── Is credit history length < 2 years?
│       ├── Yes → Higher default risk
│       └── No  → Medium default risk
│
└── No
    └── Lower default risk
```

This is an illustrative decision tree, not a rule learned from your actual dataset.

Main advantage: Captures nonlinear relationships and interactions between variables. Easy to visualize and explain when the tree is small.

Main limitation: Individual decision trees can be unstable and prone to overfitting.

Your hyperparameters

Hyperparameter	Your values	What it controls
`max_depth`	3, 5, 8, 10, 15, None	Maximum number of levels in the tree.
`min_samples_split`	2, 5, 10, 20	Minimum observations required to split a node.
`min_samples_leaf`	1, 5, 10, 20	Minimum observations required in a terminal leaf.
`criterion`	gini, entropy	Method used to measure node impurity.

`max_depth`

Controls how complex the tree can become.

- 3: Relatively simple tree.
- 5: More decision rules.
- 15: Potentially complex tree.
- None: No explicit maximum depth.

Smaller values generally reduce overfitting, but can cause underfitting.

`min_samples_split`

Determines how many observations a node must contain before the algorithm is allowed to split it.

For example:

- 2: A node containing two observations can be considered for splitting.
- 20: At least 20 observations are required.

Higher values generally make the tree more conservative.

`min_samples_leaf`

Controls the minimum number of observations allowed in each terminal node.

This is particularly important for credit-risk models because it prevents the tree from creating extremely small groups of customers based on isolated patterns.

For example, with `min_samples_leaf=10`, a terminal node must contain at least 10 training observations.

`criterion`

Defines how the algorithm evaluates the quality of a split.

- `gini` : Measures the probability of incorrectly classifying a randomly selected observation if its class is assigned according to the node's class distribution.
- `entropy` : Measures uncertainty in the class distribution using information theory.

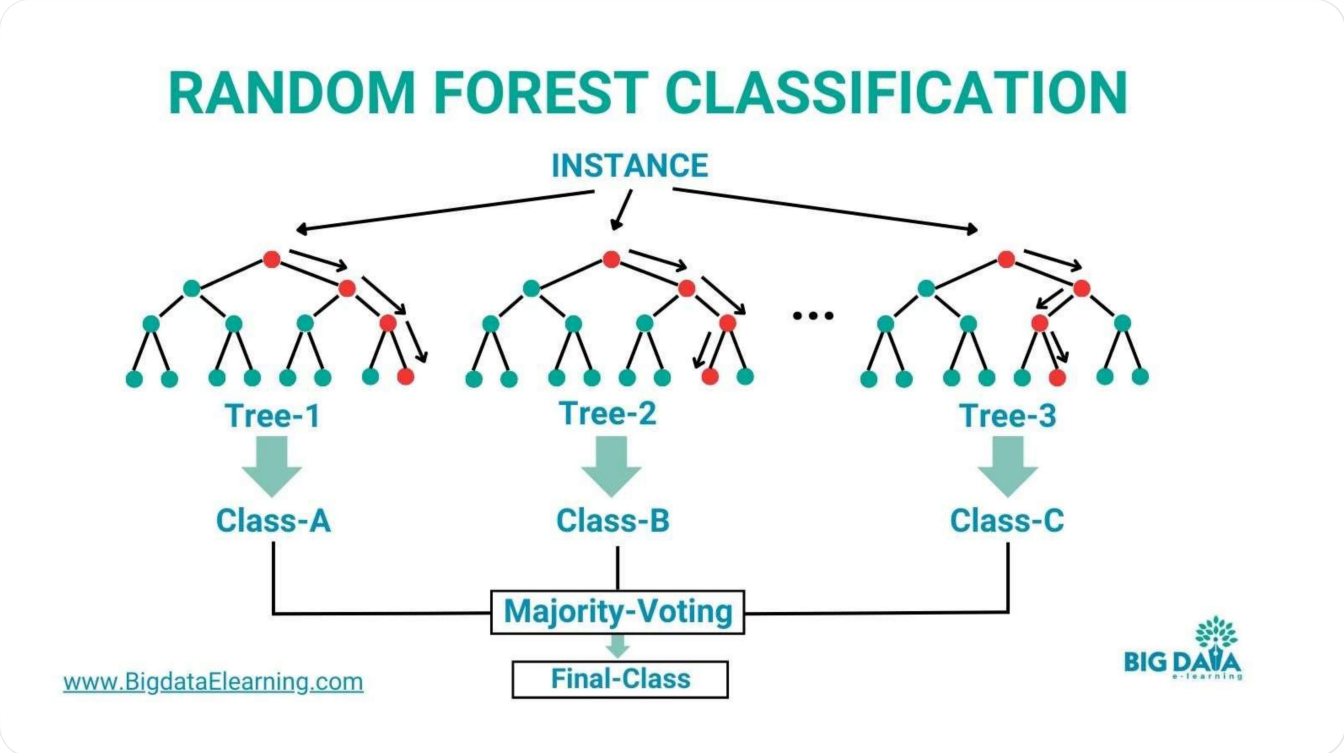
Both are legitimate choices. Their practical performance is often similar.

What you didn't include:

- `max_features` : Number of features considered when evaluating a split. Worth testing, but not essential for your initial Decision Tree comparison.
- `max_leaf_nodes` : Limits the total number of terminal nodes. Could be an alternative way to control complexity.
- `ccp_alpha` : Controls post-pruning. This is worth considering in a more advanced experiment.

I would prioritize `ccp_alpha` over adding many more parameters to the initial search.

3. Random Forest



Random Forest combines multiple Decision Trees.

Each tree is trained using a random sample of observations and a random subset of features at each split.

The final classification is based on the collective predictions of the trees.

For probability prediction, the forest aggregates the probabilities estimated by individual trees.

Main advantage: Usually more stable than a single Decision Tree and capable of capturing nonlinear relationships and feature interactions.

Main limitation: Less interpretable than an individual tree and potentially computationally expensive.

Your hyperparameters

Hyperparameter	Your values	What it controls
`n_estimators`	100, 200, 300	Number of trees.
`max_depth`	5, 10, 15, None	Maximum depth of each tree.
`min_samples_split`	2, 5, 10	Minimum observations required to split a node.
`min_samples_leaf`	1, 2, 5	Minimum observations in a terminal leaf.
`max_features`	sqrt, log2	Number of features considered at each split.

`n_estimators`

Controls how many trees participate in the ensemble.

- 100 trees: Smaller ensemble.
- 300 trees: Larger ensemble.

More trees can improve stability, but increase training time and memory consumption.

After a certain point, additional trees may provide very little improvement.

`max_depth`, `min_samples_split`, and `min_samples_leaf`

They have the same fundamental meanings as in Decision Tree.

However, here they control the complexity of every individual tree in the forest.

`max_features`

This parameter is particularly important in Random Forest.

It controls how many features are randomly considered when the algorithm searches for the best split.

- `sqrt` : Square root of the total number of features.
- `log2` : Base-2 logarithm of the total number of features.

For example, with 16 input features:

- `sqrt` considers 4 features.
- `log2` considers 4 features.

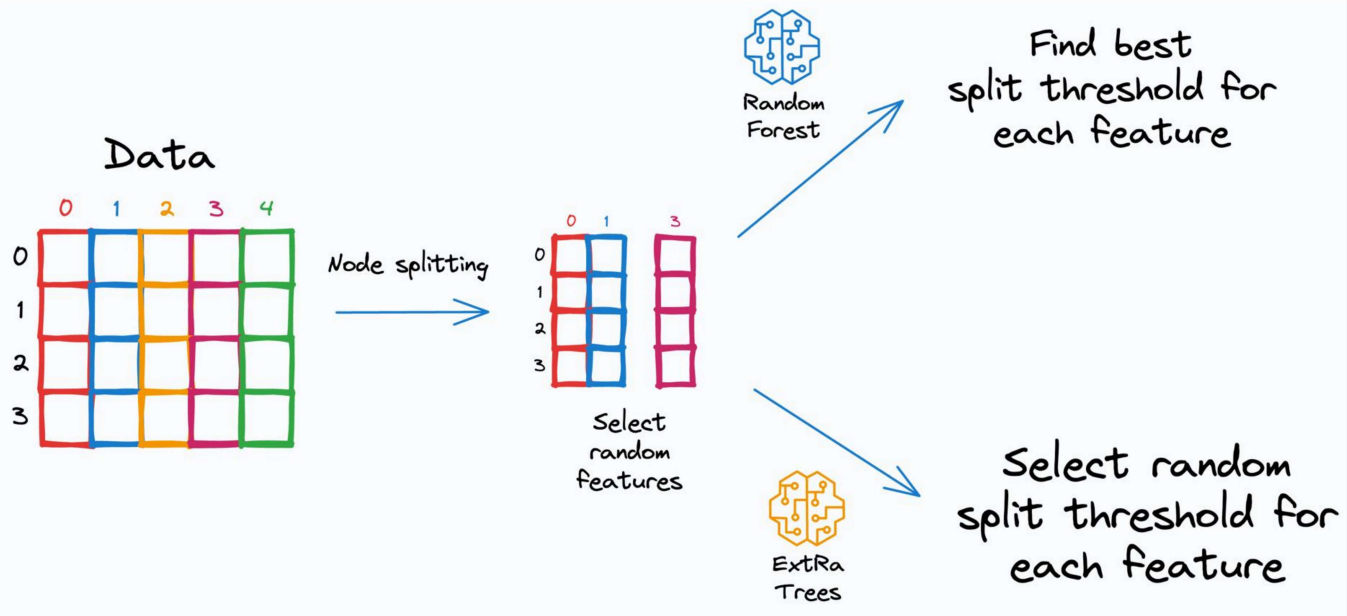
This introduces diversity among trees, which helps reduce correlation between them.

What you didn't include:

- `bootstrap` : Whether trees use bootstrap samples. The default is `True` , and I would keep it.
- `max_samples` : Controls the size of each bootstrap sample. Worth testing if you want to explore sampling diversity.
- `criterion` : You are using the default `gini` . You could test `entropy` or `log_loss` , but I wouldn't prioritize it.
- `class_weight` : Already set to `"balanced"` .

One important distinction: Random Forest supports `class_weight="balanced"` , but the exact weighting behavior differs from XGBoost's `scale_pos_weight` .

4. Extra Trees — Extremely Randomized Trees



Extra Trees is similar to Random Forest, but introduces additional randomness when building the trees.

The main difference:

- Random Forest evaluates candidate split thresholds and selects the best one.
- Extra Trees randomly selects split thresholds among candidate features and then selects the best among those random splits.

This generally produces more diverse trees.

Main advantage: Can reduce variance and perform well with complex datasets. It can also be faster to train in some situations.

Main limitation: More randomness can introduce bias, so it isn't guaranteed to outperform Random Forest.

Your hyperparameters

You use exactly the same search space as Random Forest:

Hyperparameter	Your values	What it controls
`n_estimators`	100, 200, 300	Number of trees.

`max_depth`	5, 10, 15, None	Maximum depth.
`min_samples_split`	2, 5, 10	Minimum observations to split.
`min_samples_leaf`	1, 2, 5	Minimum observations per leaf.
`max_features`	sqrt, log2	Number of features considered at each split.

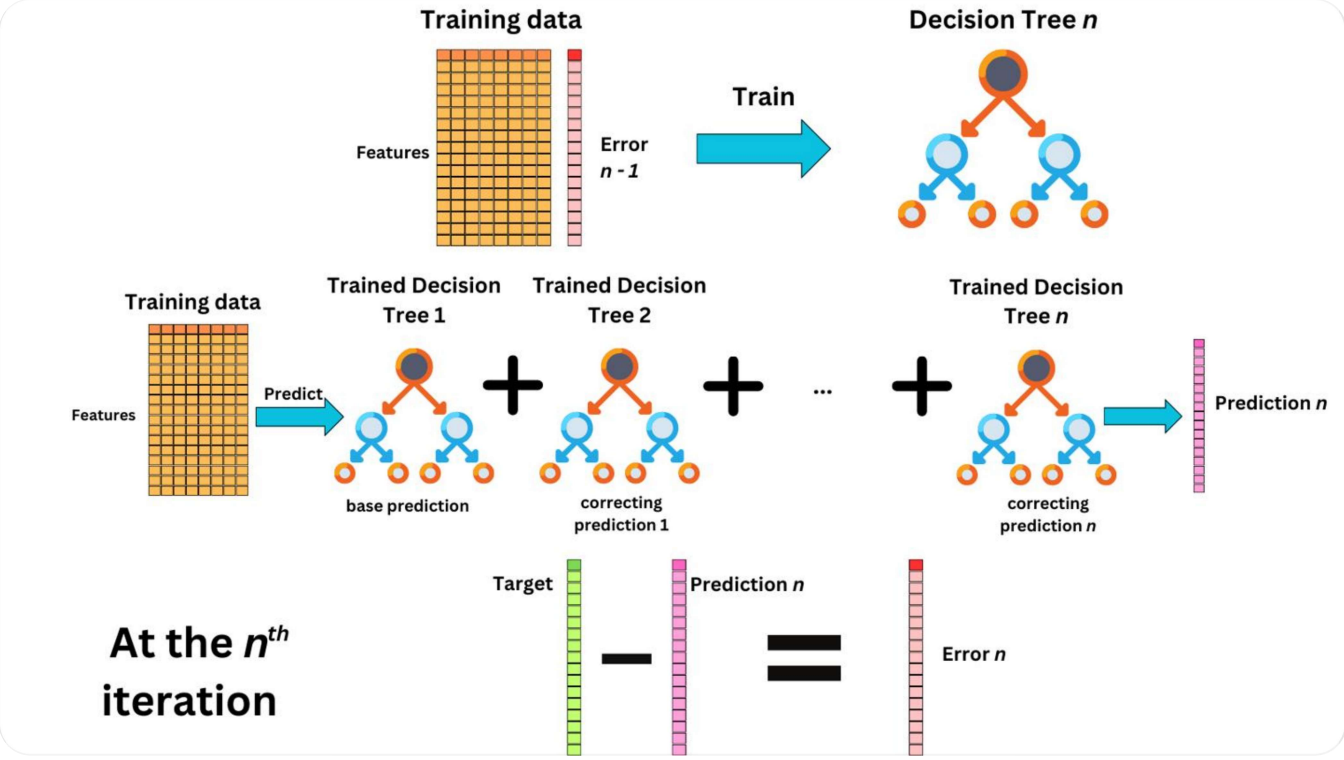
The interpretation is the same as in Random Forest.

What you didn't include:

- `bootstrap` : The default is `False` , unlike Random Forest. I would retain that default initially.
- `criterion` : The default is `gini` . You could test `entropy` or `log_loss` .
- `max_leaf_nodes` : Another way to control tree complexity.

I would keep the two models' search spaces similar initially because it makes the comparison more meaningful.

5. Gradient Boosting



Gradient Boosting is an ensemble method that builds trees sequentially.

Unlike Random Forest, where trees are trained independently, Gradient Boosting trains each new tree to correct errors made by the existing ensemble.

Imagine:

1. The first tree makes initial predictions.
2. The second tree learns from the errors of the first.
3. The third tree learns from the remaining errors.
4. The process continues until the specified number of trees is reached.

Main advantage: Often captures complex nonlinear patterns and can achieve strong predictive performance.

Main limitation: More sensitive to hyperparameter choices than Random Forest and can overfit if the boosting process becomes too aggressive.

Your hyperparameters

Hyperparameter	Your values	What it controls
`n_estimators`	100, 150, 200	Number of boosting stages (trees).
`learning_rate`	0.01, 0.05, 0.1	Contribution of each new tree.
`max_depth`	2, 3, 4	Maximum depth of each individual tree.
`min_samples_split`	2, 5, 10	Minimum observations required to split a node.
`min_samples_leaf`	1, 3, 5	Minimum observations in a terminal leaf.
`subsample`	0.8, 1.0	Proportion of observations used for each boosting stage.

`n_estimators`

Controls the number of sequential trees.

More trees can improve the model, but also increase training time and the risk of overfitting.

`learning_rate`

Controls how much each new tree contributes to the final prediction.

This is one of the most important boosting parameters.

- 0.01: Small contribution from each tree.
- 0.05: Moderate contribution.

- 0.1: Larger contribution.

There is an important relationship between `learning_rate` and `n_estimators`:

A smaller learning rate generally requires more trees to achieve comparable performance.

For example, 200 trees with a learning rate of 0.01 may learn more slowly than 200 trees with a learning rate of 0.1.

`max_depth`

Controls the complexity of each individual tree.

Gradient Boosting often uses relatively shallow trees because each tree only needs to correct part of the remaining prediction error.

Your values of 2, 3 and 4 are reasonable.

`min_samples_split` **and** `min_samples_leaf`

These control the minimum amount of data required to create new branches and terminal nodes.

They help prevent the algorithm from learning overly specific patterns.

`subsample`

Controls the fraction of training observations used for each boosting stage.

- 1.0: Uses the entire training dataset.
- 0.8: Uses 80% of the observations.

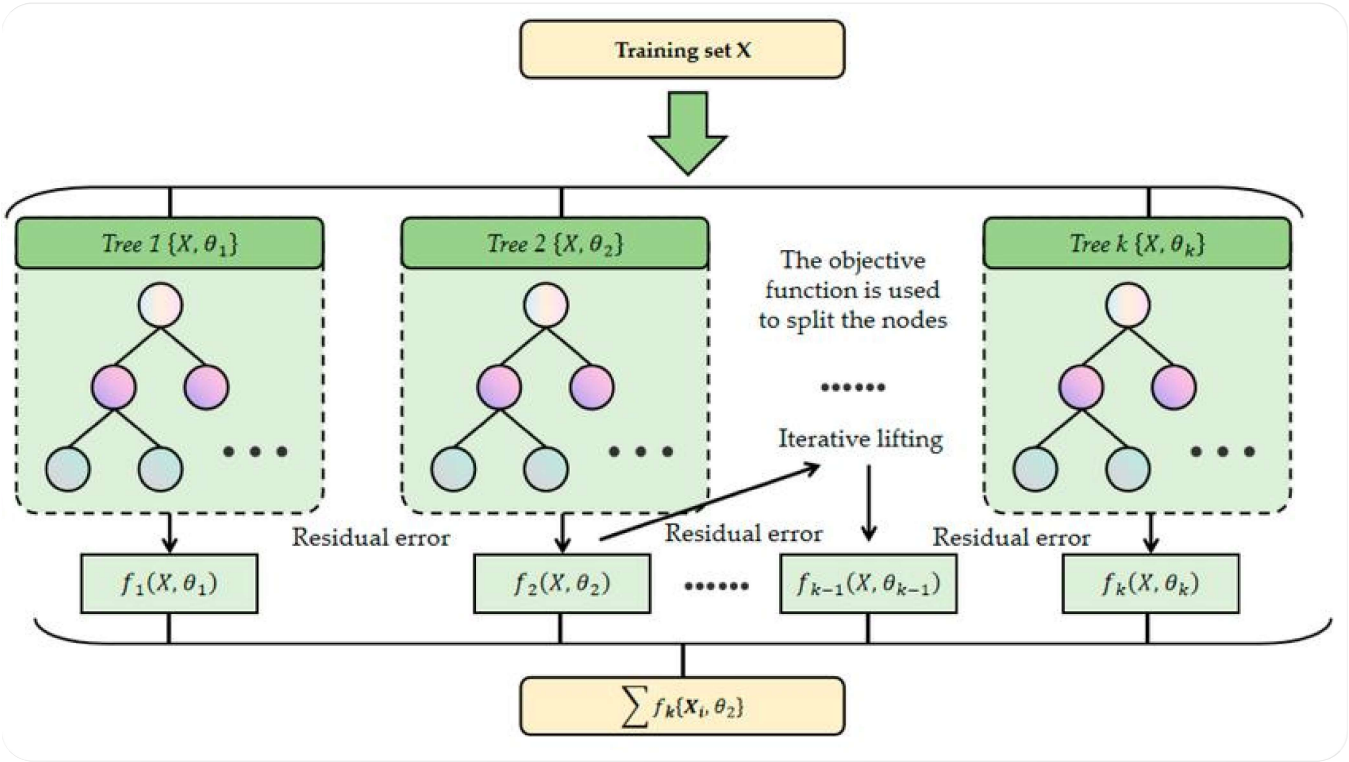
Using less than 100% introduces randomness, which can reduce variance and overfitting. This is sometimes called *stochastic gradient boosting*.

What you didn't include:

- `max_features`: Controls the number of features considered when building each tree. Worth testing.
- `max_leaf_nodes`: Alternative way to control tree complexity.
- `validation_fraction` and `n_iter_no_change`: Can enable early stopping within Gradient Boosting.
- `loss`: You are using the default `log_loss`, which is appropriate for binary classification.

I would consider early stopping in a later experiment, especially if you expand the number of estimators.

6. XGBoost — Extreme Gradient Boosting



XGBoost is an optimized and regularized implementation of gradient boosting.

It builds sequential trees, just like Gradient Boosting, but includes additional mechanisms to control complexity and improve computational efficiency.

Some important features include:

- L1 and L2 regularization.
- More sophisticated tree construction.
- Additional controls over split creation.
- Efficient handling of missing values.
- Parallelized computation.

Main advantage: Very powerful for structured/tabular datasets, such as credit-risk data.

Main limitation: Has many hyperparameters, can require careful tuning, and may overfit if its complexity isn't controlled.

Your hyperparameters

You have nine parameters in your search space.

Hyperparameter	Your values	What it controls
`n_estimators`	100, 300, 500	Number of boosting trees.
`learning_rate`	0.01, 0.04, 0.1	Contribution of each new tree.
`max_depth`	3, 5, 6	Maximum depth of each tree.
`subsample`	0.7, 0.85, 1.0	Fraction of observations used per tree.
`min_child_weight`	1, 4, 7	Minimum sum of Hessians required in a child node.
`colsample_bytree`	0.7, 0.85, 1.0	Fraction of features sampled per tree.
`gamma`	0, 0.2, 0.5	Minimum loss reduction required to create a split.
`reg_alpha`	0, 0.5, 1	L1 regularization.
`reg_lambda`	1, 4, 10	L2 regularization.

Let's look at the parameters that are more specific to XGBoost.

`min_child_weight`

This parameter controls how much evidence is required before XGBoost creates a new child node.

Technically, it represents the minimum sum of Hessians (second-order derivatives of the loss function) required in a child node.

In practical terms:

- Lower values allow more splits.
- Higher values make the model more conservative.

This is particularly useful for controlling overfitting.

`colsample_bytree`

Controls the percentage of features randomly sampled when building each tree.

For example:

- 0.7: Each tree uses 70% of the available features.

- 1.0: Each tree can use all features.

This introduces diversity and can reduce overfitting.

`gamma`

Defines the minimum loss reduction required to create a new split.

- 0: No additional minimum reduction requirement.
- 0.5: A split must improve the objective by at least 0.5.

Higher values make tree growth more conservative.

`reg_alpha` **— L1 regularization**

Penalizes the absolute values of leaf weights.

It can encourage some leaf weights to become zero, effectively simplifying the model.

`reg_lambda` **— L2 regularization**

Penalizes the squared values of leaf weights.

It discourages excessively large leaf weights and helps control model complexity.

Your model configuration

```
XGBClassifier(
    objective="binary:logistic",
    eval_metric="aucpr",
    random_state=42,
    n_jobs=1
)
```

Parameter	Meaning
`objective="binary:logistic"`	Binary classification with probability output.
`eval_metric="aucpr"`	Uses area under the precision-recall curve as XGBoost's evaluation metric.
`random_state=42`	Makes random processes reproducible.
`n_jobs=1`	Uses one CPU thread inside XGBoost.

One important distinction: `eval_metric="aucpr"` does not automatically mean that your `RandomizedSearchCV` optimizes PR-AUC. The search uses whatever you specify in its `scoring` argument. For example, `scoring="average_precision"`.

The two metrics are related but not calculated identically.

What you didn't include:

There are several additional parameters worth knowing:

- `scale_pos_weight`: Controls the relative weight of positive and negative classes. Particularly relevant to your imbalanced credit-risk dataset.
- `max_delta_step`: Can make logistic classification updates more conservative, sometimes useful for highly imbalanced datasets.
- `colsample_bylevel`: Controls feature sampling at each tree level.
- `colsample_bynode`: Controls feature sampling at each individual node.
- `tree_method`: Controls the tree construction algorithm. `hist` is worth testing for larger datasets.
- `max_leaves`: Controls the maximum number of leaves per tree, particularly useful with certain growth policies.

I would prioritize `scale_pos_weight` for your next experiment. The others are more specialized and should not all be added to the search space at once.

2. Why some hyperparameters are not included in your search

There are three main reasons why I wouldn't search every available parameter.

Reason 1: Some parameters are already set to appropriate defaults

For example:

- Logistic Regression: `penalty="l2"`.
- Random Forest: `bootstrap=True`.
- Extra Trees: `bootstrap=False`.
- Gradient Boosting: `loss="log_loss"`.
- XGBoost: `objective="binary:logistic"`.

Defaults are not necessarily optimal, but they provide a reasonable starting point.

Reason 2: Some parameters are less influential than others

For example, changing `max_depth` or `learning_rate` can substantially affect model behavior.

Changing an optimization tolerance or a less commonly used sampling parameter may have a much smaller impact.

I would prioritize parameters that directly control:

1. Model complexity.
2. Regularization.
3. Learning speed.
4. Class imbalance.
5. Sampling diversity.

Reason 3: Computational cost

Your XGBoost search space contains:

$$3^9 = 19,683 \text{ combinations}$$

With five-fold cross-validation, exhaustive GridSearchCV would require:

$$19,683 \times 5 = 98,415 \text{ fits}$$

And that excludes the final refit.

This is why RandomizedSearchCV is a sensible choice for your project.

You can investigate the most influential parameters without testing every possible combination.

3. What I would change or add to your current configuration

Considering that your best result so far is XGBoost with 72.6% Recall, I would not radically change all six models. I would make targeted improvements.

Here is my recommendation:

Model	What I would add or change	Priority
Logistic Regression	Test `C` more finely; retain L2 initially	Medium
Decision Tree	Add `ccp_alpha` for pruning	Low

Random Forest	Test `max_samples` and possibly `criterion`	Medium
Extra Trees	Test `criterion`; retain its different sampling behavior	Low
Gradient Boosting	Test `max_features` and early stopping	Medium
XGBoost	Add `scale_pos_weight`; consider `max_delta_step`	High

My suggested next XGBoost experiment

I would start with this configuration:

```
# Calculate class imbalance ratio
scale_pos_weight = (
    (y_train == 0).sum() /
    (y_train == 1).sum()
)

models["XGBoost"]["params"] = {
    "model__n_estimators": [100, 300, 500],
    "model__learning_rate": [0.01, 0.04, 0.1],
    "model__max_depth": [3, 5, 6],

    "model__subsample": [0.7, 0.85, 1.0],
    "model__min_child_weight": [1, 4, 7],
    "model__colsample_bytree": [0.7, 0.85, 1.0],

    "model__gamma": [0, 0.2, 0.5],
    "model__reg_alpha": [0, 0.5, 1],
    "model__reg_lambda": [1, 4, 10],

    "model__scale_pos_weight": [
        1,
        scale_pos_weight * 0.5,
        scale_pos_weight,
        scale_pos_weight * 1.5
    ]
}
```

This adds class-imbalance weighting without removing the other parameters.

A couple of precautions:

- Calculate the ratio using only the training data, not the complete dataset.

- If using cross-validation, consider that a fixed ratio calculated before CV is an approximation. Ideally, calculate it separately inside each training fold.
- Don't assume the highest `scale_pos_weight` will produce the best Recall. It can also increase false positives.

I would also consider testing `max_delta_step`, but only after understanding the results of this experiment.

4. An important point about your current model comparison

I would pay particular attention to how you compare these algorithms.

For a fair comparison, all models should use:

- The same training and test datasets.
- The same cross-validation strategy.
- The same primary scoring metric.
- The same preprocessing logic where appropriate.
- The same evaluation threshold when comparing classification metrics.

For your project, I would use:

```
scoring="average_precision"
```

during hyperparameter optimization.

Then, after selecting the best configuration, evaluate Recall, Precision, F1, ROC-AUC and PR-AUC on the validation/test datasets.

Remember that `class_weight="balanced"` in Logistic Regression, Decision Tree and Random Forest does not automatically make their predicted probabilities well calibrated. The same concern applies to `scale_pos_weight` in XGBoost.

5. My overall assessment

Your current model selection is quite good for a learning project because it covers different approaches:

Baseline and interpretability

Baseline **Logistic Regression**

Useful for understanding the relationships between customer characteristics and default risk.

Decision rules

Interpretability **Decision Tree**

Useful for visualizing customer risk patterns.

Ensemble learning

Ensemble **Random Forest and Extra Trees**

Useful for capturing nonlinear relationships and comparing different approaches to tree aggregation.

Boosting

Priority **Gradient Boosting and XGBoost**

Particularly interesting for structured financial datasets and potentially strong predictive performance.

My suggestion is to resist the temptation to add dozens of hyperparameters immediately.

First, understand which parameters have the greatest influence on your results. Then experiment systematically.

The next step I would take is to create a hyperparameter experiment report for XGBoost, comparing the effects of `max_depth`, `learning_rate`, `scale_pos_weight` and `min_child_weight` on Recall, Precision, PR-AUC and ROC-AUC.

That would help you understand not only which configuration performs best, but *why* it performs best.

Se quiser, posso:

- Explain how to integrate class imbalance weighting in XGBoost tuning
- Suggest early stopping implementation for Gradient Boosting
- Plan a systematic hyperparameter tuning schedule for all models